GitHub Link Refusal

https://gemini.google.com/app/5e09d0b8f34754d0

User prompt: https://github.com/TomKing062/CVE-2022-38691_38692 מה זה התוכנה הזו

Response: הפרויקט הזה הוא כלי מחקר ופיתוח (Exploit / Patcher) שנועד **לעקוף את מנגנון ה-Secure Boot (אתחול Spreadtrum) לשעבר) Unisoc במכשירים המבוססים על מעבדים של חברת (מאובטח**.

התוכנה מנצלת שתי חולשות אבטחה רשמיות שהתגלו ברכיב ה-BootROM של המעבדים הללו, המסומנות כמזהי **CVE-2022-38691** ו-**CVE-2022-38692**.

מה התוכנה עושה בפועל?

המעבד מוודא שכל שלב בעליית המערכת (Chain of Trust): "במכשירי אנדרואיד מודרניים, קיימת "שרשרת אמון (Bootloader) חתום דיגיטלית על ידי היצרן לפני שהוא מריץ אותו. אם הקובץ שונה או לא חתום, המכשיר מסרב לעלות.

הכלי הזה מאפשר לבצע שינוי (Patch) **ללא חתימה דיגיטלית תקפה** לקובצי האתחול הראשוניים של המכשיר כדי להריץ אותם:

1. **עקיפת אימות חתימה ב-BootROM:** התוכנה מנצלת באג באופן שבו המעבד בודק את החתימה הקריפטוגרפית של קובצי **SPL**-ו (משמש לצריבת תוכנה במצב חירום/פלאש) **FDL1** – האתחול הראשונים שעולה ה-Secondary Program Loader (בהדלקה רגילה של המכשיר).
2. **נטרול המשך הבדיקות בשרשרת:** לאחר שהמשתמש מסיר את בדיקת החתימה מתוך קובצי ה-FDL1/SPL עצמם (בעזרת כלים משלימים שהמפתח מציין בתיעוד), הכלי הזה (`patcher`) אורז" או מתאים אותם כך שהמעבד יסכים להריץ אותם" למרות שהם אינם חתומים על ידי היצרן (Persistent) באופן קבוע.

למה משתמשים בזה?

הכלי מיועד לחוקרי אבטחה, מפתחים ומשתמשים מתקדמים שעוסקים ב-Modding למכשירי אנדרואיד סיניים או מוזלים (מבוססי Unisoc):

- **פתיחת Bootloader (Unlock):** במכשירים שבהם היצרן חסם את האפשרות הרשמית לפתוח את ה-Bootloader.
- **השגת הרשאות Root והתקנת ROM מותאם אישית:** הרצת קוד מותאם אישית כבר משלב עליית המערכת.
- **תיקון מכשירים תקולים (Unbrick):** שעבר מודיפיקציה כדי לצרוב מחדש מכשיר שנתקע ("לבנה") ולא שימוש בקובץ FDL1 מקבל קבצים רגילים בגלל נעילות אבטחה.

תאימות למעבדים (לפי תיעוד הפרויקט)

הפרויקט אינו עובד על כל מעבד של Unisoc, הצרובה בחומרה BootROM-אלא תלוי בגרסת ה:

דגמי מעבדים (SoC)	סטטוס תמיכה
`ud710`, `ums312` (כמו T310), `ums512` (כמו T610 / T618)	**נתמך ועובד**
`sc9820e`, `sc9832e`, `sc9863a`	**פגיע, אך טרם נכתב קוד תמיכה**
`udx710` (בשל מגבלות מיקומי זיכרון ב-Stack)	**פגיע, אך לא ניתן לתמיכה טכנית**
`ums9230` (כמו T606 / T616), `ums9620`, `ums9621`	**לא פגיע (חסין לחולשה)**

User prompt: 235\225 רגע זה של נוקיה?

Response: השבב `udx710` שסימנת בטבלה אינו קשור לנוקיה 225 או 235 **לא.**.

ההבדלים המדויקים בין השבבים:

- **מהו שבב `udx710`?** זהו מודם 5G ייעודי של Unisoc (שנקרא גם Ivy510), והוא נמצא בעיקר בנתבים סלולריים (5G CPE), ולא בטלפונים פשוטים, IoT, ומכשירי (דונגלים) USB מודמים סלולריים בחיבור.

- **מה יש בנוקיה 225 4G ונוקיה 235 4G?** המכשירים האלו (כולל גרסאות 2020 ו-2024 שלהם) מבוססים על שבב אחר לגמרי של אותה חברה: **Unisoc T107** (שמספרו הקטלוגי הפנימי בקוד ובכלי צריבה הוא `ums9117`).

ומה לגבי התמיכה של הכלי הזה בנוקיה 225 / 235?

שבב ה-**T107 (`ums9117`)** מריץ מערכת הפעלה סגורה וקלה של Unisoc לטלפונים פשוטים (MOCOR RTOS / S30+) ולא שמתמקד במעבדי אנדרואיד ומודמים) אנדרואיד. השבב הזה כלל אינו מופיע ברשימת המעבדים הנתמכים בפרויקט הספציפי הזה (עם שרשרת אתחול של `FDL1` ו-`SPL`).

קשוח שמונע צריבת Secure Boot הפעילה מנגנון (יצרנית נוקיה) HMD חברת T107, בנוסף, במכשירי נוקיה החדשים מבוססי (CVE-2022-38691/2) לא חתומים, ונכון להיום הפריצה הזו FDL1 קושחות מותאמות אישית או שימוש בקובצי לא פותחת אותם.

User prompt: אילו כלים או שיטות קיימים כיום לעבודה, גיבוי או עריכת קושחה במכשירי נוקיה שמבוססים על מעבד Unisoc T107 (ums9117)?

Response: עבודה עם מכשירי נוקיה המבוססים על מעבד **Unisoc T107 / T117 (`ums9117`)** — כמו נוקיה 225 4G, 235 4G, 2660 Flip, 105 4G ו-110 4G — מתחלקת לשני תחומים: **צריבה וגיבוי** (שעובדים מצוין עם הכלים הנכונים) לעומת **עריכת קושחה ושינוי מערכת**, שנתקלים במגבלות אבטחה קשיחות של HMD.

1. כלים לתקשורת, גיבוי וצריבה (Flashing & Dumping)

spd_dump (מתוך פרויקט `spreadtrum_flash`)

זהו כלי שורת פקודה (*TomKing062*) ומתוחזק גם על ידי *Ilya Kurdyukov* פותח על ידי) הכלי המרכזי והחזק ביותר בקוד פתוח של המעבד BootROM-שמתקשר ישירות עם ה Linux-ו Windows-ל (CLI).

- **מה הוא יודע לעשות ב-T107:** לשחזר מחיצות ספציפיות, (גיבוי מלא / Full Dump) לקרוא את כל המחיצות מהמכשיר ולבצע תיקון למכשיר תקול (Unbrick).

- **איך עוקפים איתו את החסימה לקריאה:** מכיוון שהמכשיר דורש קובצי אתחול חתומים דיגיטלית (`fdl1.bin` ו-`fdl2.bin`) מתוך קובץ קושחה רשמי שהודלף לרשת FDL-ע"י נוקיה כדי לאפשר גישה לזיכרון, השיטה היא לחלץ את קובצי ה (של אותו דגם או דגם זהה חומרתית `.pac`) ולהזריק אותם לזיכרון ה RAM דרך `spd_dump`. ברגע שהמעבד מזהה שה-FDL חתום כראוי, הוא פותח גישה לקריאה וכתיבה של מחיצות הזיכרון.

SPD ResearchDownload / UpgradeDownload

התוכנה הרשמית של Unisoc לצריבת קובצי `.pac` מלאים.

- משמשת לעדכון גרסה (Downgrade) שנמוך, או החלפת אזור שפה (למשל, צריבת קושחה עם עברית על מכשיר שהגיע מלא עברית), כל עוד קובץ ה-`.pac` הוא רשמי וחתום על ידי HMD.

קופסאות ודונגלים לטכנאים (כלי פרימיום)

כוללים תמיכה מובנית במעבדי **Hydra Tool** או **Pandora Box (Z3X)**, **CM2 (Chinese Miracle-2 SP2)** כלים מסחריים כמו (חתומים בשרתים שלהם Loader הם מחזיקים מאגר קובצי FDL1/FDL2 הם חוסכים את הצורך לחפש ידנית קובצי `ums9117`. ומאפשרים איפוס קוד נעילה, גיבוי מלא וצריבה בלחיצת כפתור.

2. כלים לחילוץ, מחקר ועריכת קושחה (Unpacking & Modding)

הקושחה אינה בנויה כמו (Series 30+ שנמכרת תחת השם השיווקי) **MOCOR RTOS** מערכת ההפעלה במכשירים אלו היא Spreadtrum: אנדרואיד רגיל, אלא ארוזה במבנים ייחודיים של

- **חילוץ קובצי PAC:** ניתן לפרק את קובץ ה-`.pac` כגון) למחיצות הבודדות שלו `nor_fdl1.bin`, `kernel.bin`, `user.bin`, `mmires.bin`) באמצעות סקריפטים בפייתון לחילוץ PAC, או פשוט לפתוח את הקובץ בתוכנת

בזמן שהתוכנה פתוחה (`ImageFiles`) ולהעתיק את הקבצים המחולצים מתיקיית ה-Temp ה *ResearchDownload*.

- **`bzpwork` ו-`fphelper_t117`:** כלים ייעודיים מהמאגר של *Ilya Kurdyukov* ב-GitHub. הכלי `bzpwork` משמש לפירוק קובץ BZP), ו-`fphelper_t117` בלוקים מסוג Spreadtrum) ופריסה של דחיסת הנתונים הפנימית של קושחות ומנתח את ה-`kernel.bin` כדי לטעון את הקושחה לתוך תוכנות הנדסה (Base Addresses) ומוצא את כתובות הזיכרון המדויקות בארכיטקטורת ARMv7-R Little Endian). לאחור כמו **IDA Pro** או **Ghidra**.
- **עריכת משאבים וגרפיקה (`mmires` / `user`):** מחיצות המשאבים כוללות את הפונטים, התרגומים, הצלצולים והאייקונים של Spreadtrum. ניתן לסרוק ולחלץ אותן באמצעות כלי חילוץ משאבים של **Binwalk** המערכת.

3. המכשול המרכזי: מה אפשר ומה אי אפשר לעשות?

לעיתים כבוי, ואפשר אפילו להריץ עליהם קוד חיצוני כמו Secure Boot-שבהם ה) של יצרנים סיניים גנריים T107 בניגוד למכשירי Doom הפורט של המשחק (`fpdoom`-ב), **מופעל ומוקשח Secure Boot-מנגנון ה (HMD) במכשירי נוקיה**:

1. **מה כן עובד:**
   - גיבוי מלא של המכשיר (Dump) למחשב דרך `spd_dump`.
   - חילוץ קובצי המערכת למחשב כדי לשלוף מהם גרפיקה, צלילים, שפות או לנתח את הקוד.
   - או שחזור גיבוי מקורי (`user` / `userdata`) כתיבה מחדש של מחיצת נתוני המשתמש.
   - של המכשיר, **TA**-לפי מספר ה) מכל אזור/שפה שמתאים לדגם החומרה המדויק (`.pac`) צריבת קושחה רשמית מלאה למשל `TA-1480`).
2. **מה נחסם (הקיר של החתימה הדיגיטלית):**
   - כדי לשנות תכונות במערכת ההפעלה או למחוק (`kernel.bin` / `ps`) אם תערוך את קובץ המערכת הראשי אפליקציות מובנות ותצרוב אותו חזרה למכשיר — **המכשיר לא ידלק (Soft Brick)**.
   - שכוללת חתימת (Data Header Table Block) **DHTB** הסיבה היא שהמחיצות הקריטיות עטופות בכותרת שנקראת ה-, (כמו זו שראית קודם למעבדי אנדרואיד) `ums9117` למעבד BootROM מכיוון שאין כיום פריצת RSA של HMD. בודק את החתימה בכל הדלקה ומסרב להעלות קובץ שעבר שינוי אפילו של ביט אחד, עד שתצרוב חזרה Bootloader את המחיצה המקורית והחתומה.

איך מכניסים את המכשיר למצב צריבה (Boot Mode)?

כדי שהמחשב יזהה את המכשיר כ-`SPRD U2S Diag` (`1782:4` מזהה חומרה `d00`) עבור `spd_dump`:

1. מכבים את המכשיר ומוציאים את הסוללה (במכשירים שבהם היא נשלפת).
2. מדובר בלחיצה רצופה על **כוכבית (*)** או שילוב של T107 הייעודי לדגם — ברוב דגמי נוקיה **מקש ה-Boot** מחזיקים את העליונים Softkeys-**כוכבית + מקש ניתוק אדום**, ובחלק מהדגמים החדשים אחד ממקשי הניווט או ה.
3. למחשב (ומכניסים את הסוללה אם היא בחוץ) USB-תוך כדי החזקת המקש, מחברים את כבל ה.

User prompt: רגע ואין בכלל דרך למכשירי HMD

Response: בעקבות השאלה שלך בדקתי שוב לעומק את העדכונים האחרונים בקוד של המפתחים **דווקא יש דרך!** (Ilya Kurdyukov ו-TomKing062), ומסתבר שנמצאה פרצה ייעודית שעוקפת את ה-Secure Boot גם במכשירי נוקיה (HMD) מבוססי **UMS9117 (T107)**.

כיום, בשתי רמות HMD הנה תמונת המצב המדויקת של איך עוקפים את החסימה של:

1. עקיפה דרך חיבור USB (הרצת קוד לא חתום וגישה מלאה לזיכרון)

בזמן חיבור למחשב HMD כדי לעקוף את דרישת החתימה של FDL פתוח לגמרי, לגבות בלי קובץ `custom_fdl` למשל כדי להריץ) פקודת ניצול חולשה ייעודית `spd_dump` המפתחים הוסיפו לכלי ,(RAM-רשמי של נוקיה, או להריץ תוכנות/משחקים ישירות מה למעבד T107/T117:

הפקודה נקראת: `t117_exec_dist`

-שדורסת את מצביע החזרה בזיכרון ה ,CVE-2022-38694-בדומה ל) UMS9117-של ה BootROM-היא מנצלת חולשה ב
BootROM מוסיפים את הדגל ,HMD על מכשיר נוקיה של `spd_dump` כשמריצים את .(על ידי חישוב המרחק בין הבאפרים שלו
FDL-הזה **לפני** טעינת ה:

```
./spd_dump t117_exec_dist 0x314d fdl nor_fdl1.bin 0x40004000 ...
```

(מותאם אישית ולא חתום FDL עם קובץ `0x6200`בחלק מהמקרים הטעינה מתבצעת לכתובת `0`).

HMD? מה זה נותן לך במיידי על מכשירי

- **אתה לא כבול למגבלות של הקובץ המקורי של נוקיה. אפשר לקרוא (Unsigned FDL) מותאם אישית FDL הרצת קובץ:** ולבצע שינויים במחיצות ,eFuses ולכתוב לכל כתובת בזיכרון הפלאש, לקרוא.
- **או אפילו פרויקט (NES/אמולטורים של גיים-בוי ,Doom הרצת) FPDoom פרויקטים כמו :RAM-הרצת קוד חיצוני ב** Nokia 3210 4G כמו UMS9117 מלאה על מכשירי נוקיה מבוססי Alpine Linux שמריץ מערכת הפעלה) **fplinux** באמצעות השיטה הזו HMD עובדים על מכשירי (ו-225/235.

2. עקיפה קבועה / עריכת קושחה במכשיר עצמו (`sdboot_t117` ו-Dummy Signature)

כך שהשינוי (NOR/NAND) מה קורה אם אתה רוצה לערוך את הקושחה עצמה של הטלפון ולצרוב אותה בחזרה לזיכרון הפלאש
?יישאר גם כשהטלפון מנותק מהמחשב

בתהליך הפיתוח שלו התגלו שני דברים .(`fpdoom` בתוך פרויקט) **`sdboot_t117`** קיים כלי בשם Ilya Kurdyukov במאגר של
UMS9117-קריטיים לגבי מעבד ה:

1. **טריק ה-Dummy Signature (חתימת דמה):** בחלק מהמחיצות ובשלבים מסוימים בשרשרת העלייה של UMS9117, של (Free Space) המערכת בודקת שקיים בלוק חתימה תקני במבנה שלו, או שניתן להזריק קוד לתוך השטח הפנוי שעוברת את הבדיקה של (`dummy signature`) הקושחה מבלי לשבור את האימות הראשוני, או להשתמש בחתימת דמה ה-Bootloader הפנימי.
2. **עלייה מכרטיס זיכרון (SD Boot):** בתוך השטח (משני Bootloader) יודע "להשתיל" קטע קוד זעיר `sdboot_t117` הכלי הפנוי בקושחה המקורית של הטלפון ולצרוב אותה חזרה.
    - עולה כרגיל (S30+) כשהטלפון נדלק רגיל – מערכת ההפעלה של נוקיה.
    - אם מדליקים את הטלפון תוך כדי לחיצה על מקש מוגדר – הקוד שהושתל נכנס לפעולה וטוען מערכת הפעלה חלופית, MicroSD-ישירות מתוך כרטיס ה (`fpbin/fpmain.bin`) תפריט אפליקציות או משחקים בלי שום בדיקת חתימה!

ומה אם רוצים רק "להכשיר" קושחה של נוקיה (למשל למחוק דפדפן, משחקים או תפריטים)?

אם המטרה שלך היא לערוך את מערכת ההפעלה המקורית של נוקיה (למשל, להסיר את האייקון של הדפדפן/פייסבוק או לשנות
טקסטים ותרגומים):

- **מחיצת המשאבים (`mmires` / `userdata`):** בחלק ניכר ממכשירי נוקיה מבוססי T107, מחיצת המשאבים הגרפיים והשפות הראשי Bootloader-קשיחה כמו ה RSA או מחיצת הגדרות המשתמש אינן מאומתות באותה רמת חתימת (`nor_fdl` / `bootloader`), או שהבדיקה שלהן היא רק Checksum / CRC. פשוט שאפשר לעדכן אחרי העריכה.
- **עריכת ה-Kernel (`ps` / `kernel`):** אם עורכים את הקוד של המערכת עצמה, צריבה רגילה תיכשל אם לא מעדכנים את בדיוק כמו מה) של המכשיר Bootloader-או משתמשים בפאטצ'ר שמנטרל את בדיקת החתימה בשלב ה Header-כותרת ה `ums9117`-עושה באנדרואיד, ועבורו נפתח דיון ספציפי להתאמתו ל CVE-2022-38691-ש `t117_exec_dist`).

User prompt: > 1. עקיפה דרך חיבור USB (הרצת קוד לא חתום וגישה מלאה לזיכרון) כדי לעקוף את דרישת החתימה של < רשמי של נוקיה, או להריץ FDL פתוח לגמרי, לגבות בלי קובץ custom_fdl למשל כדי להריץ) בזמן חיבור למחשב HMD T107/T117: > > פקודת ניצול חולשה ייעודית למעבד spd_dump המפתחים הוסיפו לכלי ,(RAM-תוכנות/משחקים ישירות מה הפקודה נקראת: t117_exec_dist > < היא מנצלת חולשה ב BootROM-של ה UMS9117) בדומה ל-CVE-2022-38694, על מכשיר spd_dump כשמריצים את .(על ידי חישוב המרחק בין הבאפרים שלו BootROM-שדורסת את מצביע החזרה בזיכרון ה FDL: > > Bash > ./spd_dump t117_exec_dist 0x314d fdl מוסיפים את הדגל הזה לפני טעינת ה-HMD, נוקיה של nor_fdl1.bin 0x40004000 ... > (0 בחלק מהמקרים הטעינה מתבצעת לכתובת `0x6200` עם קובץ FDL מותאם אישית ולא חתום).

אתה לא כבול למגבלות של (Unsigned FDL): מותאם אישית FDL הרצת קובץ < < מה זה נותן לך במיידי על מכשירי < < HMD? ולבצע שינויים במחיצות. < < הרצת eFuses, הקובץ המקורי של נוקיה. אפשר לקרוא ולכתוב לכל כתובת בזיכרון הפלאש, לקרוא שמריץ) fplinux או אפילו פרויקט (NES/אמולטורים של גיים-בוי, Doom הרצת) FPDoom פרויקטים כמו :RAM-קוד חיצוני ב עובדים על מכשירי (ו-Nokia 3210 4G 225/235 כמו UMS9117 מלאה על מכשירי נוקיה מבוססי Alpine Linux מערכת הפעלה מה < (sdboot_t117 ו-Dummy Signature) באמצעות השיטה הזו. < < 2. עקיפה קבועה / עריכת קושחה במכשיר עצמו HMD כך שהשינוי יישאר (NOR/NAND) קורה אם אתה רוצה לערוך את הקושחה עצמה של הטלפון ולצרוב אותה בחזרה לזיכרון הפלאש בתהליך. (fpdoom בתוך פרויקט) sdboot_t117 קיים כלי בשם Ilya Kurdyukov גם כשהטלפון מנותק מהמחשב? < < במאגר של בחלק :(חתימת דמה) Dummy Signature-טריק ה < < :UMS9117-הפיתוח שלו התגלו שני דברים קריטיים לגבי מעבד ה המערכת בודקת שקיים בלוק חתימה תקני במבנה שלו, או שניתן UMS9117, מהמחיצות ובשלבים מסוימים בשרשרת העלייה של של הקושחה מבלי לשבור את האימות הראשוני, או להשתמש בחתימת דמה (Free Space) להזריק קוד לתוך השטח הפנוי הכלי :(SD Boot) הפנימי. < < עלייה מכרטיס זיכרון Bootloader-שעוברת את הבדיקה של ה (dummy signature) בתוך השטח הפנוי בקושחה המקורית של הטלפון ולצרוב אותה (משני Bootloader) יודע "להשתיל" קטע קוד זעיר sdboot_t117 עולה כרגיל. < < אם מדליקים את הטלפון תוך כדי לחיצה (+S30) חזרה. < < כשהטלפון נדלק רגיל – מערכת ההפעלה של נוקיה (fpbin/fpmain.bin) על מקש מוגדר – הקוד שהושתל נכנס לפעולה וטוען מערכת הפעלה חלופית, תפריט אפליקציות או משחקים בלי שום בדיקת חתימה! אז אני יכול לעשות תוכנה שלמה מחדש בשביל הנוקיה, MicroSD-ישירות מתוך כרטיס ה

שתרוץ (C / C++ בשפת) **כן, בהחלט.** בזכות הפרצות והכלים האלו, אתה יכול לכתוב תוכנה או מערכת משלך מאפס Response: או אפילו להרים עליו מערכת הפעלה מבוססת לינוקס (Bare-Metal), ישירות על החומרה של הנוקיה.

כדי להבין מה באמת אפשרי ואיפה המגבלות, כדאי להפריד בין **כתיבת אפליקציה/מערכת עצמאית** לבין **החלפת מערכת הטלפוניה המקורית של נוקיה**:

1. מה אתה יכול לבנות ולעשות ב-100% כבר עכשיו?

באמצעות התשתית של `fplinux`, אתה מקבל גישה ישירה לחומרה של המכשיר, מסך, (של Ilya Kurdyukov) או `fpdoom` (ודרייברים `minimal libc`) המפתחים כבר בנו ספריית בסיס **(Bare-Metal ב-C):** פיתוח תוכנות, ממשקים ומשחקים ללא שום תלות במערכת ההפעלה המקורית של נוקיה (RAM מקשים, כרטיס זיכרון, שמע וזיכרון:

- למעבד `ums9117`. אתה כותב קוד ב-C, מקמפל אותו עם הקומפיילר של ARM (מתוך ה-Android NDK או `gcc-arm-none-eabi`), והופך אותו לקובץ `.bin`. הקוד שלך מקבל שליטה מלאה על:
  - **המסך (Framebuffer):** ציור פיקסלים, חלונות, טקסטים בעברית או גרפיקה ישירות לבקר המסך (למשל `ST7789` ברזולוציית 240x320).
  - **לוח המקשים (Keypad Matrix):** קריאת לחיצות מקשים בזמן אמת.
  - **מערכת קבצים (FAT32):** קריאה וכתיבה של קבצים, מסדי נתונים או תמונות מתוך כרטיס ה-MicroSD.
  - **שמע (Audio):** השמעת צלילים ומוזיקה דרך הרמקול או האוזניות.
- **הרצת לינוקס מלא (פרויקט `fplinux`):** מעבד ה-T107 (`ums9117`) מכיל ליבת **ARM Cortex-A7** בתדר 1.0GHz וזיכרון RAM של כ-48MB עד 64MB (בניגוד לדור הישן שהכיל רק 4MB). **לינוקס אמיתי (Alpine Linux)**. ברמת החומרה — אתה יכול להריץ סקריפטים, כלי C-ברגע שיש לך לינוקס שרץ מכרטיס הזיכרון, אתה לא מוגבל רק ל שורת פקודה, או ממשקים גרפיים קלים שנכתבו ללינוקס.

2. שתי דרכים להריץ את התוכנה שלך על המכשיר

שיטת הרצה	איך זה עובד	יתרונות וחסרונות
דרך USB (לפיתוח ובדיקות)	מחברים את הטלפון למחשב ומריצים את הקובץ בעזרת `spd_dump` והפרצה ל-RAM שבנית ישירות `t117_exec_dist 0x314d`.	**אפס סיכון למכשיר.** לא נוגעים בזיכרון הפנימי (Flash). ברגע שמוציאים סוללה ומדליקים, הטלפון חוזר להיות נוקיה רגיל. מעולה לבדיקת קוד תוך שניות.
דרך כרטיס SD (`sdboot_t117`)	צורבים התאמה קטנה (עם או מכינים כרטיס SD כשמדליקים את הטלפון עם Bootloader ב-MBR). מקש לחוץ, הוא טוען את התוכנה שלך מכרטיס הזיכרון (`fpmain.bin`).	**נייד ועצמאי.** מאפשר לך להשתמש בטלפון כרגיל לשיחות (כשמדליקים רגיל), או להיכנס למערכת/תוכנה שבנית (כשמדליקים עם מקש הקיצור).

3. הקאץ' היחיד: מודם הסלולר (SMS-שיחות ו)

אם המטרה שלך היא לבנות **תוכנה ייעודית** (למשל: נגן מוזיקה משוכלל, יומן/מנהל משימות, קורא ספרים, מחשבון/כלי עבודה, או משחק) — הכל פתוח בפניך ויש לך קוד מקור מוכן להתבסס עליו.

אבל אם המטרה שלך היא **להחליף לגמרי את מערכת ההפעלה של הטלפון** כך שהמערכת *שלך* גם תוציא ותקבל שיחות טלפון ו-SMS ב-4G (VoLTE):

- הוא סגור וקנייני Unisoc הרכיב שמנהל את הרשת הסלולרית במעבד של ה (משולב עמוק בתוך RTOS-המקורי של ה MOCOR וקובצי ה DSP/Modem-).
- אתה עוקף את מערכת ההפעלה של נוקיה, ולכן המודם (`fplinux`), או בלינוקס Bare-Metal כשאתה מריץ קוד משלך ב (מול המודם IPC-אלא אם מבצעים הנדסה לאחור מורכבת מאוד לממשק ה) הסלולרי אינו מופעל במצב הזה.
- המערכת של נוקיה נשארת בשביל שיחות :SD-דרך כרטיס ה **Dual-Boot (הפעלה כפולה)** לכן, הפתרון הפרקטי ביותר הוא רגילות, והתוכנה שלך עולה מכרטיס הזיכרון מתי שאתה רוצה להשתמש בה.

איך מתחילים בפועל?

זוהי אפליקציית — `fptest` כולל תיקייה בשם `fpdoom` אתה לא צריך לכתוב את הדרייברים למסך ולמקשים מאפס. הפרויקט של שרק מאתחלת את המסך של הנוקיה, מדפיסה טקסט ומגיבה למקשים. לוקחים את C קטנה ונקייה בשפת (Template) שלד `fptest`, מוחקים את הבדיקות שלה, וכותבים בפנים את הלוגיקה והממשק של התוכנה שאתה רוצה.

User prompt: רגע אני רוצה להתקין גרסה אחת חכמה שאני אוכל נגיד בתקיה יעודית בכרטיס זיכרון לשים קוד ואז כשאני מפעיל הוא סורק את התקיה ואז הוא מוסיף את הסמל לדף הבית של התוכנה ששם או תוכנות ואפש להריץ אותם...

Response: מה שאתה מתאר הוא בעצם **מערכת הפעלה מינימלית** שיושבת על הטלפון, **/ (משגר אפליקציות דינמי) Launcher** קוראת ממנה את התוכנות והאייקונים שלהן, ומציגה אותן על (למשל `SD:/apps/`) ובכל הדלקה סורקת תיקייה בכרטיס הזיכרון המסך כאילו זה "דף בית" של סמארטפון.

החדשות המעולות: אתה לא רק יכול לעשות את זה – **התשתית לזה כבר קיימת בחלקה הגדול בתוך הפרויקט של `fpdoom` (תחת השם `fpmenu`)**, ואפשר לשדרג אותה בדיוק למה שאתה רוצה.

איך זה עובד כיום ב-`fpmenu` ואיך משדרגים את זה לחזון שלך?

בפרויקט של Ilya Kurdyukov יש רכיב שנקרא **`fpmenu`**. כשמתקינים את פריצת ה-`sdboot_t117` על המכשיר, הקובץ הראשי שהטלפון מריץ מכרטיס הזיכרון נקרא `fpbin/fpmain.bin`.

1. **המצב הקיים (`fpmenu`):** כרגע, `fpmenu` הוא תפריט טקסטואלי פשוט שקורא קובץ הגדרות (`fpbin/config.txt`) של `.bin`-הוא טוען את קובץ ה – OK מכרטיס הזיכרון, מציג רשימה של תוכנות/משחקים, וכשאתה בוחר אחד מהם ולוחץ ומריץ אותו RAM-אותה תוכנה לזיכרון ה.
2. **השדרוג למה שאתה רוצה (סריקה אוטומטית + אייקונים בדף בית):** במקום להסתמך על קובץ טקסט ידני ותפריט רשימה כך שיעבוד כמו משגר חכם (קובץ אחד קצר וברור, C-שכתוב ב) `fpmenu` משעמם, אנחנו עורכים את הקוד של:
   - **מבנה התיקיות בכרטיס הזיכרון:** תיצור בכרטיס הזיכרון תיקייה בשם `apps`. כל תוכנה חדשה שתרצה להוסיף לטלפון, תהיה פשוט תיקייה חדשה בפנים, למשל:
     - `SD:/apps/calculator/` (ובתוכה: `app.bin`, `icon.bmp`, `info.txt`)
     - `SD:/apps/notes/` (ובתוכה: `app.bin`, `icon.bmp`, `info.txt`)
   - **סריקה אוטומטית בהפעלה:** לספריית ה-FAT32 של הפרויקט יש פונקציות מובנות לסריקת תיקיות (`fat_read_dir`). ה-Launcher שהמערכת עולה, ה-`/apps/` שולף, החכם עובר בלולאה על כל התיקיות בתוך `icon.bmp`, ומצייר על המסך **רשת של אייקונים** (**למשל 3x3 של Grid 3**) עם סמן שזז מכל אחת את השם ואת קובץ ה ביניהם בעזרת מקשי החיצים של הנוקיה.
   - **הרצה בלחיצת כפתור:** כשלוחצים על סמל של תוכנה, ה-Launcher מעתיק את `app.bin` שלה מתוך הכרטיס אל RAM-כתובת פנויה ב (Jump) ומעביר אליה את הפיקוד (!RAM-עשרות מגה-בייטים פנויים ב T107 יש לנוקיה).

רוצה משהו עוד יותר מטורף? להריץ קוד בלי לקמפל! (Lua / JavaScript)

שמת לב שכתבת: *"לשים קוד ואז כשאני מפעיל הוא סורק..."* אם אתה מתכוון שאתה רוצה לזרוק לכרטיס הזיכרון **קובצי קוד טקסטואליים (סקריפטים)** שאפשר לערוך אפילו ב"פנקס רשימות" בלי להסתבך עם קומפיילרים של C בכל פעם:

QuickJS / Duktape (זעיר JavaScript מנוע) או **Lua** כמו C-אפשר לשלב בתוך "הגרסה החכמה" שלך מנוע שפה קליל ב:

- צורבים/מתקינים את המערכת החכמה **פעם אחת בלבד**.
- קריאת (`screen.drawText`, `screen.drawRect`) לציור על המסך JS או Lua-המערכת כוללת בתוכה פקודות מוכנות ב וקריאת קבצים (`keypad.getKey`) מקשים.
- מעכשיו, כדי ליצור תוכנה חדשה לנוקיה, אתה רק פותח תיקייה חדשה בכרטיס הזיכרון, שם בה תמונת אייקון וכותב קובץ טקסט פשוט (`main.lua` או `main.js`).
- ברגע שתכניס את הכרטיס לנוקיה ותדליק – האייקון יופיע בדף הבית, ולחיצה עליו תריץ את הסקריפט שלך מיד!

איך מתקינים את זה בפועל על הנוקיה?

התהליך מתחלק לשני שלבים חד-פעמיים:

1. **פתיחת הדלת בטלפון (חד-פעמי):** מחברים את הנוקיה למחשב, מגבים את הקושחה שלו עם `spd_dump`, ומשתילים בה זה לוקח בדיוק כמה דקות. מאותו רגע, הטלפון שלך יודע לעלות מכרטיס ה- (`sdboot_t117`) את הקפיצה לכרטיס הזיכרון SD.
2. **בניית ה"מערכת החכמה" (`fpmain.bin`):** מוסיפים לו את התצוגה הגרפית של `fpmenu`, לוקחים את קוד המקור של ושמים את הקובץ המוכן ,(ואם תרצה – גם מנוע הרצת סקריפטים) `/apps/` האייקונים (דף הבית) ואת סריקת תיקיית בכרטיס הזיכרון.

User prompt: א האם נשאר גם התוכנה המקורית של הטלפון (אני לא רוצה שהוא יעלה גם מהכרטיס וגם ישאר על הטלפון את ....הישן אלא רק אחת על הטלפון (או על הכרטיס אבל פחות מומלץ) וכמו כן האם אני להוריד את התוכנות המקוריות

כאן בדיוק נמצאת הנקודה הכי קריטית שצריך להבין לפני שבוחרים באיזה כיוון ללכת, כי יש הבדל עצום בין **להחליף** Response: **לגמרי את המערכת של נוקיה** לבין **לערוך ולהגמיש את המערכת המקורית של נוקיה**.

התשובות הישירות לשתי השאלות שלך:

1. האם אפשר שהתוכנה שלך תהיה היחידה על הטלפון (בלי המערכת הישנה ובלי כרטיס SD)?

טכנית – כן. אתה יכול לצרוב את התוכנה שלך ישירות לזיכרון הפנימי של הטלפון (Flash) במקום הקושחה של נוקיה, והיא תעלה אוטומטית בכל הדלקה בלי שום כרטיס זיכרון ובלי זכר למערכת הישנה.

אבל יש לזה מחיר כבד מאוד (בעיית השיחות והקליטה): במערכת המקורית של נוקיה (MOCOR RTOS), הקוד שמצייר את **בלוטות' וניהול טעינה ,SMS הודעות ,Gשיחות, קליטת 4)** התפריטים והתוכנות המקוריות והקוד שמנהל את **המודם הסלולרי** מקומפלים יחד לתוך קובץ בינארי אחד גדול (`kernel` / `ps`).

- אם תמחק את המערכת המקורית של נוקיה ותתקין במקומה מערכת חדשה מאפס – **הטלפון לא יוכל להוציא או לקבל SMS-שיחות ו**, כי למערכת החדשה שלך לא יהיו הדרייברים הסגורים של המודם הסלולרי של Unisoc. המכשיר יהפוך או נגן משוכלל בלי קליטה סלולרית (PDA) "בעצם ל"מחשב כף יד.
- אם אתה רוצה שהטלפון **ימשיך לתפקד כטלפון לשיחות**, הקושחה המקורית של נוקיה *חייבת* להישאר בזיכרון הפנימי.

2. האם אפשר להוריד (למחוק) את התוכנות המקוריות של נוקיה?

כן, וזה כנראה הכיוון שאתה באמת מחפש! אם אתה רוצה טלפון שעובד כרגיל לשיחות, בלי צורך לעשות "הפעלה כפולה" (Dual Boot), אבל שיהיה נקי מהתוכנות המקוריות המעצבנות של נוקיה (כמו דפדפן אינטרנט, משחקי ניסיון, פייסבוק, רדיו וכו') ובמקומן יריץ את התוכנות שלך – עושים **Modding** שנמצאת על הטלפון **לקושחה המקורית**:

1. **הסרת תוכנות מקוריות מהתפריט של נוקיה:** שואבים את הקושחה מהטלפון למחשב. בתוך הקושחה יש טבלת תפריטים (`mmires`). באמצעות עריכת הקושחה (פירוק משאבים / Hex), אפשר **להעלים ולנתק** ומחיצת משאבים (Menu Tree) **מהתפריט הראשי** כל אפליקציה מובנית שאתה לא רוצה (משחקים, דפדפן, נגן וכו'), כך שהמשתמש לעולם לא יראה אותן ולא יוכל להיכנס אליהן.

2. **הפיכת אחד הסמלים בתפריט של נוקיה ל"משגר החכם" שלך (In-OS Hooking):** במקום שהמשגר החכם יעלה *במקום* את הקוד שלך לתוך אחת הפונקציות של הקושחה המקורית (למשל, להחליף את (Hook) הטלפון בהדלקה, אפשר לחבר הפעולה של כפתור "משחקים" או "תוספות" בתפריט המקורי).

    - הטלפון נדלק רגיל לתוך המערכת של נוקיה (שערכנו וניקינו מתוכנות מיותרות), ויש לו שיחות ו-SMS כרגיל.
    - כשאתה נכנס בתפריט ל"תוכנות שלי" (או לוחץ על קיצור מקשים), המערכת קופצת לקוד שלך, סורקת את התיקייה הייעודית) ומציגה את התוכנות שהוספת, (הפנימית של הטלפון Storage-בכרטיס הזיכרון או במחיצת ה.

השוואה קצרה כדי שתבחר מה מתאים לך:

תכונה	אפשרות א': מערכת חדשה לגמרי על הטלפון	אפשרות ב': עריכת המערכת המקורית של נוקיה + הזרקת המשגר שלך
מה נשאר מהישן?	כלום. מוחקים הכל וכותבים מערכת מאפס.	ליבת הטלפון נשארת, אך מוחקים/מסתירים את התוכנות המובנות שלא רוצים.
שיחות ו-SMS (סלולר)	❌ **לא עובד** (אין דרייברים למודם 4G).	✅ **עובד ב-100%** כרגיל.
איפה זה מותקן?	על הזיכרון הפנימי של הטלפון.	על הזיכרון הפנימי של הטלפון (התוכנות הנוספות נטענות מתיקייה).
רמת קושי	בינונית (אבל צריך C-יש שלד מוכן ב לבנות הכל).	גבוהה יותר (דורש הנדסה לאחור ב-Ghidra לקובץ הקושחה של הדגם שלך).

איזה משני הכיוונים האלו קולע למה שאתה מתכנן – מכשיר עם שיחות סלולריות שבו אנחנו מנקים את המערכת של נוקיה ומשתילים בתוכה את סורק התוכנות שלך, או מכשיר ללא סלולר שמריץ אך ורק מערכת עצמאית שלך?

User prompt: אבל למה אני לא יכול להעתיק את הקרנל ? בשביל הדרייברים?

Response: שמריצים) הסיבה שאתה לא יכול פשוט "להעתיק את הקרנל" לתוך מערכת חדשה משלך היא שבמכשירים האלו **אין הפרדה בין הקרנל, הדרייברים והתוכנות כמו שיש ב-**MOCOR RTOS של Unisoc), מערכת הפעלה בזמן אמת בשם **Windows, לינוקס או אנדרואיד.**

כדי להבין למה, כדאי להכיר איך הקושחה של הנוקיה בנויה מבפנים:

1. הכל "מותך" לקובץ אחד סגור (Monolithic RTOS Blob)

כמו) והתוכנות הן קבצים נפרדים, (כמו במחשב או באנדרואיד, הקרנל הוא קובץ אחד, הדרייברים הם קבצים נפרדים `.ko` או `.sys`. בנוקיה T107 זה לא עובד ככה: `.exe` או `.apk`).

- **(ThreadX RTOS)** + יצרו את הגרסה, הקומפיילר שלהם לקח את הקוד של ה**קרנל** בזמן שהמהנדסים של נוקיה/Unisoc **כל התוכנות המובנות (דפדפן, (MMI)** + ה**דרייברים של המודם והמסך** + **ממשק המשתמש והתפריטים של נוקיה** — וחיבר **משחקים, שעון)** (Linked) את כולם יחד לקובץ בינארי אחד ענק ודחוס.
- בתוך הקובץ הזה אין שמות של פונקציות (הכל נמחק בקימפול), אין קובצי דרייברים נפרדים שאפשר לשלוף, והכל קורא להכל באמצעות כתובות זיכרון קבועות מראש (Hardcoded memory addresses).

אם אתה מעתיק את הקובץ הזה ("הקרנל") למערכת שלך ומריץ אותו – אתה בעצם מריץ את כל מערכת ההפעלה המקורית של נוקיה, כי הן אותו קובץ בדיוק.

2. בעיית ה-RTOS לעומת לינוקס

הנפרדות שיש בטלפון (שמכילות את הקושחה של מעבד התקשורת המשני), הן לא Modem/DSP-אפילו אם תיקח את מחיצות ה המקורית תנהל אותן דרך זיכרון RTOS-הן מצפות שמערכת ה `AT+CALL` מתקשרות עם המעבד הראשי בפקודות פשוטות כמו שאין להם שום תיעוד פומבי. מפתחי פרויקט Unisoc עם מאות מבני נתונים סודיים של (Shared Memory / SIPC) משותף

`fplinux` מנסים כבר חודשים לפענח איך לדבר עם המודם הזה מתוך לינוקס, וזה עדיין לא עובד (שהרימו לינוקס על הנוקיה הזה) לשיחות.

3. הפתרון החכם: במקום שאתה תיקח את הקרנל – תן לקרנל שלהם להריץ אותך! (Hijacking)

מכיוון שהקרנל, הדרייברים והתוכנות של נוקיה תקועים יחד באותו קובץ, החוקרים עושים טריק הפוך שנותן לך **בדיוק את התוצאה שרצית**:

במקום לבנות מערכת מאפס ולנסות להעתיק אליה דרייברים, **משאירים את הקובץ המקורי של נוקיה, אבל "עוקרים" מתוכו את דף הבית והתוכנות המובנות, ומחברים את הקוד שלך ישירות לפונקציות שכבר קיימות בקרנל שלהם!**

איך זה עובד בפועל?

1. **כל הדרייברים כבר עובדים בשבילך:** המערכת של נוקיה עולה תוך שנייה, מאתחלת את המודם (Gיש שיחות ו-4), מאתחלת את המסך, הבלוטות', הסוללה והשמע.
2. **נטרול התוכנות של נוקיה:** בקובץ הקושחה מוצאים את טבלת התפריטים או את פונקציית ציור מסך הבית, ומשנים כמה בייטים (Patch) מבוטלות לחלוטין כך שהתוכנות המקוריות של נוקיה (משחקים, אינטרנט וכו') לא יפריעו.
3. **שימוש בדרייברים הפנימיים שלהם:** בתוך הקובץ של נוקיה כבר קיימות פונקציות מוכנות שמדברות עם החומרה (למשל פונקציה שמציירת על המסך, פונקציה שפותחת קובץ בכרטיס הזיכרון, ואפילו פונקציה שמוציאה שיחת טלפון!). כשאתה מזריק את ה"משגר החכם" שלך לתוך המערכת שלהם, הקוד שלך פשוט קורא לכתובות הזיכרון של הפונקציות האלו בקרנל של נוקיה.

במילים אחרות: **הקרנל של נוקיה נשאר למטה בתור "מנוע" שמחזיק את הדרייברים והשיחות, והקוד שלך מתלבש מעליו בתור הממשק שסורק את התיקייה ומריץ את התוכנות שלך.**

User prompt: רגע אז אני יכול לעשות נגיד שכפתור תוספים יעבור לכל התוכנות שיש לי בכרטיס...

Response: **בדיוק! קלעת בול.** זה הפתרון הכי אלגנטי, יציב וחכם שאפשר לעשות במכשירים האלו.

שעון מעורר, אנשי קשר, מהתפריט הראשי כל זכר לדפדפן או למשחקים המובנים, וכשאתה לוחץ על כפתור "תוספים" (או כל סמל אחר שתבחר בתפריט) – הטלפון קופץ לקוד שלך, סורק את התיקייה בכרטיס הזיכרון ומציג את כל התוכנות ששמת שם עם האייקונים שלהן.

איך הקסם הזה עובד מאחורי הקלעים? (Function Hook-שיטת ה)

בתוך הקושחה המקורית של נוקיה יש טבלה שמחברת בין כל סמל בתפריט הראשי לבין הפונקציה בקוד שמופעלת כשלוחצים עליו. לכל פריט בתפריט יש מזהה (Menu ID), וכתובת זיכרון, אייקון, טקסט ("תוספים") מריץ (Pointer) של הפונקציה שהוא מריץ.

אנחנו עושים שינוי כירורגי קטן בקושחה שנמצאת על הטלפון:

1. **מוצאים את הכפתור בקושחה:** מאתרים בקובץ הקושחה את השורה בטבלה שמייצגת את כפתור "תוספים" (או למשל "סנייק" משחק מובנה כמו).
2. **משנים את הכתובת (Redirect):** במקום שהכפתור יצביע לפונקציה המקורית של נוקיה, אנחנו משנים את הכתובת שלו כך שיצביע אל קטע קוד קטן שלנו שהזרקנו לתוך השטח הפנוי בזיכרון הפנימי של הטלפון (Free Space).
3. **מה הקוד הקטן בטלפון עושה?** תפקידו היחיד של הקוד שהזרקנו לטלפון (ששוקל בקושי 2–4 קילו-בייט) הוא לגשת לכרטיס הזיכרון, לטעון מתוכו לזיכרון ה-RAM קובץ בשם `SD:/system/launcher.bin` ולהריץ אותו.
4. **המשגר החכם שלך נכנס לפעולה:** הקובץ `launcher.bin` שיושב בכרטיס הזיכרון, ולכן אתה יכול לעדכן ולשדרג אותו) קורא את השמות והאייקונים של `SD:/apps/`, עולה על המסך, סורק את תיקיית התוכנות ששמת שם, ומצייר אותן על המסך. כשאתה לוחץ על כפתור "חזור / ניתוק", הוא מחזיר את השליטה לתפריט הרגיל של נוקיה (מתי שתרצה בלי לצרוב שוב את הטלפון!).

למה השיטה הזו גאונית בשבילך?

- **צורבים את הטלפון רק פעם אחת בחיים:** ברגע ששינית את החיבור של כפתור "תוספים" (והעלמת את התוכנות המיותרות של נוקיה), סיימת להתעסק עם צריבות מסוכנות לטלפון.

- **הכל נשלט מכרטיס הזיכרון:** גם ה"משגר החכם" עצמו וגם כל התוכנות החדשות שלך יושבים כקבצים רגילים בכרטיס הזיכרון. רוצה לעצב מחדש את דף הבית של התוספים? להוסיף תוכנה חדשה? לתקן באג? אתה פשוט מחבר את הכרטיס למחשב, מעתיק את הקובץ לתיקייה, ולוחץ שוב על "תוספים" בטלפון.

- **אפס איבוד של יכולות הטלפון:** אם באמצע שאתה משתמש בתוכנה שלך בכרטיס הזיכרון נכנסת שיחת טלפון – הקרנל של נוקיה עדיין חי ברקע או שתוכל לצאת בלחיצה אחת ולענות לשיחה.

איך עושים את זה בפועל (4 שלבים ברורים)

1. **שאיבת הקושחה (Dump):** שואבים את הקושחה של הנוקיה שלך למחשב באמצעות `spd_dump` והפרצה `t117_exec_dist`.

2. **פריסת הקושחה:** מריצים את הכלי `bzpwork` כדי לפרוס את הקוד הדחוס של המערכת (`kernel.bin`), ומשתמשים ב-`fphelper_t117` כדי לקבל את מפת הזיכרון.

3. **איתור כפתור "תוספים" והסתרת תוכנות מיותרות:** פותחים את הקובץ ב-Ghidra (או בעזרת סקריפט חיפוש בפייתון), מוצאים את טבלת התפריטים, מבטלים את הסמלים שאתה רוצה להוריד (כמו דפדפן/משחקים), ומשנים את המצביע של "תוספים" לקוד הטעינה שלנו.

4. **בניית ה-Launcher והתוכנות ב-C:** בכרטיס הזיכרון ומציג אייקונים, `apps` שסורק את תיקיית Launcher-מקמפלים את ה ושמים אותו בכרטיס.

והאם כבר שאבת ממנו, (כתוב מתחת לסוללה TA למשל 225 או 235, ואיזה מספר) איזה דגם מדויק של נוקיה יש לך כרגע ביד? גיבוי של הקושחה למחשב

User prompt: והאם אפשר לחבר את הלינוקס לקרמל?

Response: בצורה שבה אתה חושב על זה (שהלינוקס והקרנל של נוקיה יעבדו יחד במקביל וידברו זה עם זה) – **לא, אי אפשר לשלב אותם יחד בזמן אמת.** אבל **כן אפשר להפעיל את הלינוקס מתוך כפתור "תוספים"** בתור מערכת שעולה כשאתה צריך אותה.

כדי להבין למה אי אפשר "למזג" אותם ולמה זה עובד רק בתור מעבר מאחד לשני, הנה ההסבר הטכני הפשוט:

1. למה לינוקס והקרנל של נוקיה לא יכולים לרוץ יחד? (מונופול על החומרה)

מערכת הפעלה כמו **Linux** במעבד **UMS9117 (T107)** (**Single-core ARM Cortex-A7**). יש ליבת עיבוד ראשית אחת בלבד היא לא "תוכנה רגילה" שרצה מעל מערכת קיימת – לינוקס *הוא בעצמו קרנל*.

ברגע שאתה מפעיל את הלינוקס (`fplinux`):

1. הקרנל של לינוקס לוקח שליטה בלעדית על המעבד, על טבלת ניהול הזיכרון (MMU) ועל הפסיקות (Interrupts) של החומרה.

2. **את הקרנל של נוקיה** שהיה שם קודם **דורס ומוחק מזיכרון ה-RAM** כדי לעלות, לינוקס.

3. מרגע שלינוקס עלה – הקרנל של נוקיה כבר לא קיים בזיכרון. הוא לא יכול לרוץ ברקע, לינוקס לא יכול "לקרוא לפונקציות שלו", וכתוצאה מכך גם הקשר עם המודם הסלולרי מתנתק כל עוד אתה נמצא בתוך לינוקס.

2. מה כן אפשר לעשות? (קפיצה ללינוקס מתוך התפריט וחזרה)

למרות שהם לא יכולים לחיות יחד *באותו רגע* בזיכרון, אתה בהחלט יכול לחבר את טעינת הלינוקס לכפתור **"תוספים"** (או שיהיה אחד האייקונים בתוך המשגר החכם שלך):

- **איך זה עובד בכניסה:** אתה משתמש בטלפון של נוקיה כרגיל (עם שיחות והכל). כשבא לך להשתמש בלינוקס, אתה נכנס ל"תוספים" ולוחץ על האייקון של לינוקס. הקוד הקטן שלנו טוען את קרנל הלינוקס מכרטיס הזיכרון ומעביר אליו את ה-RAM. השליטה. תוך כמה שניות המסך מתחלף ויש לך לינוקס עובד על הנוקיה.

- המכשיר מבצע אתחול מהיר (שלוקח כ-3–4 שניות בנוקיה) וחוזר מיד למערכת הטלפון הרגילה של נוקיה עם הקליטה הסלולרית. **איך חוזרים לטלפון:** כשסיימת לעבוד בלינוקס, אתה בוחר "יציאה / Reboot".

3. אז מה עדיף לך: תוכנות ב-C/סקריפטים או לינוקס?

זו ההחלטה הכי חשובה לתכנון שלך, והנה ההבדל המעשי ביניהם על הנוקיה:

תכונה	משגר חכם שלנו (תוכנות ב-C / סקריפטים)	מעבר ללינוקס מלא (`fplinux`)
מה קורה לקרנל של נוקיה?	**לא** – הקרנל של נוקיה נמחק מה-RAM. כדי לחזור לטלפון חייבים **צריך לעשות הפעלה מחדש** כדי לחזור לטלפון!	או לרוץ כתהליך בתוכו – אפשר להקפיא אותו ב-RAM. כדי לחזור לטלפון חייבים Reboot.
שימוש בדרייברים של נוקיה	✅ **אפשרי!** הקוד שלך יכול לקרוא לפונקציות שכבר קיימות בקרנל של נוקיה (קריאת קבצים, מסך, ואפילו פונקציות מערכת).	❌ **בלתי אפשרי.** לינוקס חייב דרייברים משלו שנכתבו במיוחד ללינוקס.
מהירות פתיחת תוכנה	**מיידי (0.1 שניות).** לוחצים על האייקון והתוכנה על המסך.	**איטי יותר (כ-5 עד 10 שניות)** עד שהקרנל של לינוקס עולה מהכרטיס.
למה זה מתאים?	לתוכנות יומיומיות מהירות: מחשבון, פתקים, סידור/ספרים, משחקים, נגן, כלי עזר שמרגישים כמו חלק טבעי מהטלפון.	להרצת תוכנות לינוקס כבדות יחסית, טרמינל אמולטורים כבדים או כלי פיתוח של לינוקס.

שיושב על כפתור "תוספים" וסורק את התיקייה **C-השורה התחתונה:** אתה יכול ליהנות משניהם! אם נבנה את **המשגר החכם ב** בכרטיס הזיכרון – רוב התוכנות שתשים בתיקייה ייפתחו מיד ובחלקות בלי להפיל את הקרנל של נוקיה, ואחת התיקיות בכרטיס שרק אם תלחץ עליה, היא תטען את קרנל הלינוקס המלא – "Linux" הזיכרון יכולה להיות.

User prompt: יופי נשמע טוב ואני יכול להוסיף מנועה פייתון קטן או לשדרג לתמיכה ב HTTPS?

Response: במעבד ה **כן לשניהם!**-UMS9117 (שיש לו מעבד Cortex-A7 1 מהיר של GHz ועשרות מגה-בייטים של RAM פנוי), שני הדברים האלו אפשריים בהחלט — אבל לכל אחד מהם יש סוד טכני קטן שחשוב להכיר כדי שהוא יעבוד חלק.

1. הוספת מנוע פייתון קטן (MicroPython)

יהיה כבד מדי ללא מערכת הפעלה מלאה, אבל יש פתרון שתפור בדיוק על (CPython) להריץ את הפייתון הרגיל של המחשב המידות של המעבד הזה: **MicroPython** או מנוע עוד יותר זעיר שנקרא **PikaPython**).

- **מה זה MicroPython?** זהו מנוע פייתון שלם (Python 3, תומך בתחביר של 'מחלקות, רשימות, מילונים, קבצים וכו) נקי ומתקמפל לקובץ בינארי ששוקל **פחות מ-300 קילו-בייט!** הוא מיועד מראש לרוץ על שבבים ללא מערכת C-שכתוב כולו ב כמו של הנוקיה RTOS או בתוך (Bare-Metal) הפעלה.
- **איך זה יעבוד במשגר החכם שלך?**
  1. שלך ב Launcher-C פעם אחת יחד עם ה MicroPython אנחנו מקמפלים את המנוע של C.
  2. אנחנו מחברים (Binding) ב-C כמה פונקציות בסיסיות של הנוקיה לתוך הפייתון: למשל מודול שנקרא `nokia` עם פקודות כמו `nokia.draw_text()`, `nokia.draw_image()`, `nokia.get_key()`.
  3. מעכשיו, בתוך תיקיית `SD:/apps/my_app/` בכרטיס הזיכרון, אתה פשוט שם קובץ טקסט רגיל בשם `main.py`!
  4. מהכרטיס ומריץ אותו מיד על המסך של `.py`-כשלוחצים על האייקון של התוכנה במשגר, המנוע קורא את קובץ ה הנוקיה – בלי לקמפל שום דבר במחשב!

2. שדרוג לתמיכה ב HTTPS (תקשורת מוצפנת מודרנית)

כאן יש נקודה סופר מעניינת: אחת הבעיות הכי גדולות בטלפונים פשוטים (וגם בדפדפן המובנה של נוקיה) היא שהם משתמשים בגרסאות הצפנה עתיקות (SSL / TLS 1.0 או תעודות אבטחה פגות תוקף וה,) ולכן רוב האתרים והAPIs המודרניים בעולם פשוט חוסמים אותם (שדורשים **TLS 1.2 / TLS 1.3**).

?אמיתי HTTPS-איך פותרים את זה ומשדרגים ל

שנקראת C-משלבים בתוך הקוד שלנו ספריית הצפנה קלה ב **mbedTLS** או **BearSSL**, ששוקלת בקושי 30 קילו-בייט של קוד!):

1. **החלק של ההצפנה (TLS 1.2 / 1.3):** טהור ורצה בקלות אדירה על המעבד של הנוקיה. אפשר C-הספרייה הזו כתובה ב MicroPython (מודול `urequests` או `ssl`), אפילו לחבר אותה ישירות ל-MicroPython: כך שתוכל לכתוב בפייתון פשוט `response = urequests.get("[https://api.example.com/data](https://api.example.com/data)")`

2. **האתגר הטכני (החיבור לרשת הסלולרית):** מכיוון שספריית ה HTTPS- צריכה צינור לתקשר דרכו עם העולם (Socket של TCP/IP), מנוהל על ידי הקרנל של נוקיה, אנחנו לא יכולים להשתמש בדרייבר רשת חיצוני, אלא צריכים להמודם של ה-4G **להתלבש על פונקציות הרשת שכבר קיימות בתוך הקרנל של נוקיה**.

   - בתוך הקושחה של נוקיה (MOCOR RTOS) קיימת מערכת Sockets פנימית (שנקראת `sci_sock_socket`, `sci_sock_connect`, `sci_sock_send`, `sci_sock_recv` – בדיוק הפונקציות שבהן הדפדפן של נוקיה משתמש כדי לגלוש).
   - על ידי מעקב, Ghidra בעזרת) ברגע שאנחנו מאתרים את הכתובות של 4 הפונקציות האלו בקושחה של הנוקיה שלך (אחרי קוד הדפדפן המקורי), אנחנו מחברים אותן כ"צינור" לתוך ספריית **mbedTLS** שלנו.

התוצאה: הקרנל של נוקיה פותח עבורך את חיבור ה-4G העתיק והשבור של SSL-הרגיל שלו, אבל במקום להשתמש במנגנון ה HTTPS-**מודרני**, ויכולה לדבר ב **mbedTLS** נוקיה, התוכנה שלך מצפינה ומפענחת את המידע בעצמה עם מאובטח עם כל שרת API או Webhook מודרני בעולם ה) אפילו עם ה-API של Gemini או צ'אטבוטים!).

איך תיראה הארכיטקטורה המלאה של הטלפון שלך:

1. **שכבת הבסיס (בזיכרון הטלפון):** הקושחה המקורית של נוקיה (שערכנו כדי להעיף ממנה תוכנות מיותרות), שמנהלת את השיחות והחיבור לאינטרנט ,4G-ה.

2. **המשגר והמנוע (`SD:/system/launcher.bin`):** קובץ אחד חזק שיושב בכרטיס הזיכרון וכולל את דף הבית עם האייקונים + מנוע **MicroPython** + ספריית **mbedTLS** ל-HTTPS.

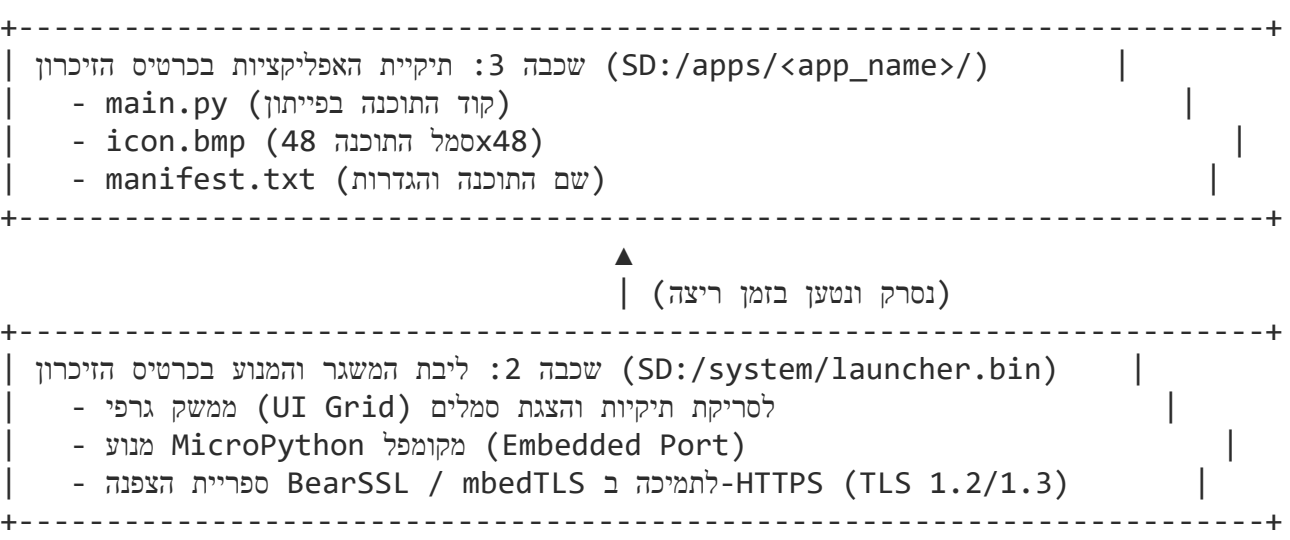
3. **תיקיית התוכנות (`SD:/apps/`):** שם אתה זורק תיקיות עם אייקון וקובץ קוד בפייתון (`main.py`), שיכול לצייר על המסך, ולשלוח בקשות HTTPS לקרוא מקשים, ולשלוח בקשות לרשת!

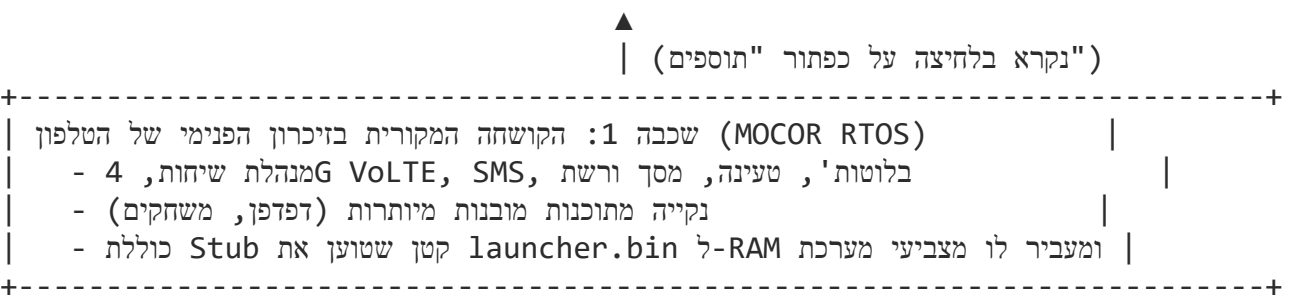
User prompt: יופי תבנה תוכנית עבודה מקיפה..

Response: תוכנית עבודה הנדסית מלאה לפיתוח והטמעה של סביבת ההרצה המותאמת **Custom Smart Launcher + MicroPython + HTTPS** עבור מכשירי נוקיה מבוססי **Unisoc T107 (`ums9117`)**, תוך שמירה מלאה על תפקוד הטלפון והמודם הסלולרי.

מבט על: ארכיטקטורת המערכת

המערכת בנויה מ-3 שכבות נפרדות כדי להבטיח שצורבים את הטלפון **פעם אחת בלבד**, וכל שאר הפיתוח והעדכונים מתבצעים בכרטיס הזיכרון:





שלב א': הכנת סביבת הפיתוח במחשב (Windows 11)

כדי לקמפל קוד למעבד של הנוקיה (ARM Cortex-A7) ולנתח את הקושחה, מכינים תיקיית עבודה מרכזית עם הכלים הבאים:

1. **כלי קומפילציה (Toolchain):**
   - הורדת **Arm GNU Toolchain** (`arm-none-eabi-gcc`) או שימוש ב-Clang מתוך **Android NDK**.
   - התקנת `make` ו-`git` דרך MSYS2 או WSL ב-Windows 11).
2. **כלי מחקר ותקשורת של Unisoc:**
   - שכפול המאגרים של Ilya Kurdyukov:
     - `spreadtrum_flash` (לגיבוי וצריבה `spd_dump` כולל את).
     - `fpdoom` (כולל את ספריות הבסיס למסך, מקשים, FAT32 ואת `fphelper_t117` ו-`bzpwork`).
3. **כלי הנדסה לאחור:**
   - **Ghidra** (חינמי) או IDA Pro – לניתוח קובץ הקושחה ומציאת כתובות הפונקציות של התפריט והרשת.

שלב ב': גיבוי וניתוח הקושחה המקורית (Reverse Engineering)

בשלב הזה לא משנים כלום בטלפון, אלא רק שואבים ממנו את המערכת וממפים אותה:

1. **גיבוי מלא (Full Dump) ביצוע:**
   - שילוב של `full_flash.bin` לקובץ NOR/NAND Flash-וגיבוי מלא של כל זיכרון ה Boot חיבור הטלפון במצב (גיבוי ביטחון למקרה של תקלה + חומר גלם לעבודה).
2. **פריסת הקושחה:**
   - הרצת `bzpwork` לחילוץ מחיצת הקוד הראשית (`ps` / `kernel`) מתוך הדחיסה של Spreadtrum.
   - הרצת `fphelper_t117` על הקובץ הפרוס כדי למצוא אוטומטית כתובות בסיס (Base Addresses), טבלאות מיפוי מקשים (`keymap`) ופונקציות מערכת מוכרות.
3. **איתור 3 נקודות המפתח ב-Ghidra:** טוענים את הקובץ הפרוס ל-Ghidra (ארכיטקטורת `ARM v7 Little Endian`) ומאתרים:
   - **טבלת התפריט הראשי (Main Menu Table):** מחפשים את מזהי הטקסט או האייקונים של התפריט הראשי כדי למצוא איפה מוגדרים הדפדפן, המשחקים וכפתור "תוספים" (`Extras`).
   - **פונקציות מערכת וזיכרון:** כתובות הפונקציות `SCI_Malloc`, `SCI_Free`, ופונקציית רענון המסך (`LCD_InvalidateRect` או ה-Framebuffer הישיר).
   - **פונקציות הרשת (Sockets):** חיפוש מחרוזות הקשורות לדפדפן או ל-TCP/IP כדי לאתר את 5 הפונקציות של הקרנל: `sci_sock_socket`, `sci_sock_connect`, `sci_sock_send`, `sci_sock_recv`, `sci_sock_socketclose`, ופונקציית ה-DNS (`sci_gethostbyname`).

שלב ג': פיתוח ה-Launcher בC (לפני הוספת הפייתון)

בונים את קובץ ה-`launcher.bin` על בסיס תשתית ה-C של `fpdoom/fptest` ובודקים אותו ישירות ב-RAM (דרך כבל USB, ללא צריבה):

1. **ניהול זיכרון וחזרה חלקה לנוקיה (Non-Destructive Execution):**
   - במקום שהקוד שלנו ידרוס את ה-RAM (או רץ באזור ה-RAM הפנוי שבקצה ה-64MB של השבב), הקוד מקצה לעצמו בלוק זיכרון פנוי של הקרנל של נוקיה, בכניסה, כך שלחיצה על מקש "יציאה/אדום" פשוט משחזרת אותם (Registers `R4-R11, LR`) שמירת מצב האוגרים וחוזרת מיד לתפריט של נוקיה בלי לעשות Reboot.
2. **סורק התיקיות (Directory Scanner):**
   - כתיבת פונקציה ב-C שפותחת את תיקיית `SD:/apps/` בכרטיס הזיכרון.
   - עבור כל תת-תיקייה שנמצאה, הפונקציה בודקת אם קיים בה קובץ `main.py` (או `app.bin`), קוראת את השם מ-`manifest.txt`, וטוענת לזיכרון את התמונה `icon.bmp`.
3. **ממשק דף הבית (Grid UI):**
   - בניית לולאת תצוגה שמציירת על המסך (240x320) רשת של 3 על 3 אייקונים, עם כותרת תחתונה בעברית/אנגלית של שם התוכנה המסומנת, וניווט עם מקשי החיצים ומקש הבחירה האמצעי.

שלב ד': הטמעת MicroPython ומנוע ה-HTTPS

ברגע שה-Launcher מציג אייקונים ועובד, מוסיפים לתוכו את "המוח":

1. **שילוב MicroPython (`ports/minimal` או `ports/embed`):**
   - הורדת קוד המקור של MicroPython וקימפולו כספרייה סטטית (`libmicropython.a`) עבור מעבד ARM Cortex-A7.
   - הגדרת גודל ה-Heap של פייתון (4MB למשל מתוך ה-RAM הפנוי).
2. **יצירת מודול ה-API של הנוקיה בפייתון (`import nokia`):** בסיסיות לתוך סביבת הפייתון כדי שכל חשיפת פונקציות C סקריפט בכרטיס הזיכרון יוכל לשלוט בטלפון:
   - `nokia.clear(color)` – ניקוי מסך.
   - `nokia.text(x, y, "שלום", color)` – כתיבת טקסט (כולל תמיכה ביישור לימין לעברית).
   - `nokia.rect(x, y, w, h, color)` / `nokia.image(x, y, "pic.bmp")` – ציור גרפיקה.
   - `nokia.update()` – רענון המסך.
   - `nokia.get_key()` – קבלת המקש שנלחץ.
3. **שילוב BearSSL / mbedTLS 4 וחיבור ל-G של הנוקיה:**
   - הוספת ספריית **BearSSL** (מומלצת כאן מאוד כי היא קומפקטית, לא דורשת הקצאות זיכרון מורכבות ותומכת ב-TLS 1.2 מודרני).
   - חיבור פונקציות ה-Low-Level I/O של BearSSL אל מצביעי הפונקציות `sci_sock_send` ו-`sci_sock_recv` שמצאנו בקרנל של נוקיה בשלב ב'.
   - חשיפת פונקציה בפייתון: `nokia.https_get(url, headers)` ו-`nokia.https_post(url, data, headers)` שמבצעת את כל תהליך ה-TLS Handshake ומחזירה את הטקסט/JSON מהשרת ישירות לקוד הפייתון שלך!

שלב ה': ניקוי הקושחה וחיבור כפתור "תוספים" (הצריבה החד-פעמית)

ניגשים לערוך את קובץ הקושחה של USB, עובדת ונבדקה דרך ה (`launcher.bin`) רק אחרי שכל המערכת בכרטיס הזיכרון הטלפון:

1. **ניקוי תוכנות מקוריות:**
    - בטבלת התפריטים שמצאנו בשלב ב', מסירים/מבטלים את הרשומות של הדפדפן המובנה, המשחקים וכל תוכנה אחרת שתרצה להוריד מהתפריט.
2. **כתיבת ה-Loader הקטן (Stub) והצמדתו לכפתור "תוספים":**
    - כותבים פונקציה זעירה ב-C/Assembly (1- מ-KB פחות) ושותלים אותה באזור ריק בקובץ הקושחה (`0xFF`).
    - הפונקציה פותחת את הקובץ `SD:/system/launcher.bin`, מעבירה אליו את ה-RAM טוענת אותו לזיכרון ומקפיצה אל הפיקוד.
    - משנים את מצביע הפונקציה של כפתור "תוספים" בטבלת התפריט כך שיצביע אל ה-Stub שלנו.
3. **אריזה וצריבה:**
    - החלת תיקון החתימה (`sdboot_t117` patch), וצריבתה דחיסת הקושחה הערוכה חזרה עם `bzpwork` (או שימוש ב-`sdboot_t117`) לטלפון.

טבלת אבני דרך (Milestones) לביצוע מסודר

שלב	משימה	תוצר סופי (Deliverable)	רמת סיכון למכשיר
1	חיבור הטלפון למחשב ושאיבת גיבוי מלא	קובץ `backup_full.bin` שמור במחשב	**אפס** (קריאה בלבד)
2	הרצת תוכנת "שלום עולם" ב-C דרך USB	מסך הנוקיה מציג טקסט שלנו ומגיב למקשים	**אפס** (רץ ב-RAM בלבד)
3	בניית ה-Launcher שסורק תיקיות בכרטיס SD	דף בית עם אייקונים שנטענים מתיקיית `SD:/apps/`	**אפס** (רץ ב-RAM בלבד)
4	קימפול MicroPython לתוך ה-Launcher	הרצת קובץ `main.py` מתוך כרטיס הזיכרון על המסך	**אפס** (רץ ב-RAM בלבד)
5	מיפוי הקושחה ב-Ghidra (תפריטים + Sockets)	רשימת כתובות זיכרון מדויקות של הדגם שלך	**אפס** (עבודה על המחשב)
6	עריכת הקושחה, חיבור כפתור "תוספים" וצריבה	טלפון נקי שמפעיל את המערכת שלך בלחיצה על "תוספים"	**נמוכה** (יש גיבוי מלא לשחזור)
